CO2 Assignment 2: C Programming Solutions

Course: Problem Solving & Programming in C **Student Name:** [Student Name] | **Roll No:** [Roll Number]

Q1 — Precedence, Associativity and the Discount Bug

1. Pre-Execution Predictions & Operator-by-Operator Evaluation

Given the original unparenthesized C statements using integer variables (`price`, `qty`, `senior` ∈ {0, 1}):

```
total = price * qty - price * qty * 15 / 100 * senior + price * qty * 5 / 100;
net   = total > 1000 ? total - 50 : total;
```

Operator Precedence & Associativity Order:

1. **Multiplicative operators (`*`, `/`):** Highest precedence among arithmetic operators here, evaluated strictly **left-to-right**. Because all operands are `int`, `/` performs *truncating integer division*.
2. **Additive operators (`+`, `-`):** Evaluated next, **left-to-right**, combining the three multiplicative sub-expressions: `(Term 1) - (Term 2) + (Term 3)`.
3. **Relational operator (`>`):** Evaluated before the ternary operator in `net`.
4. **Ternary conditional operator (`?:`):** Evaluates `total > 1000`; selects `total - 50` if true, else `total`.
5. **Assignment operator (`=`):** Lowest precedence; stores the final value into `total` and `net`.

Bill Case (price, qty, senior)	Step-by-Step C Evaluation (Integer Math)	Predicted total / net	Pharmacist Intended (Exact)
Bill 1: price = 400, qty = 3, senior = 1	T1 = 400×3 = 1200 T2 = 1200×15/100×1 = 18000/100×1 = 180 T3 = 1200×5/100 = 6000/100 = 60 total = 1200 − 180 + 60 = 1080	total = 1080 net = 1030	Gross = 1200; Disc = 180 After disc = 1020; GST (5%) = 51 total = 1071.00; net = 1021.00
Bill 2: price = 15, qty = 1, senior = 1	T1 = 15×1 = 15 T2 = 15×15/100×1 = 225/100×1 = 2 T3 = 15×5/100 = 75/100 = 0 total = 15 − 2 + 0 = 13	total = 13 net = 13	Gross = 15; Disc = 2.25 After disc = 12.75; GST (5%) = 0.6375 total = 13.39; net = 13.39
Bill 3: price = 350, qty = 3, senior = 1	T1 = 350×3 = 1050 T2 = 1050×15/100×1 = 15750/100×1 = 157 T3 = 1050×5/100 = 5250/100 = 52 total = 1050 − 157 + 52 = 945	total = 945 net = 945	Gross = 1050; Disc = 157.50 After disc = 892.50; GST (5%) = 44.625 total = 937.13; net = 937.13

2. Verification Program & Observed Output

```
#include <stdio.h>

int main(void) {
    int prices[] = {400, 15, 350}, qtys[] = {3, 1, 3}, seniors[] = {1, 1, 1};
    for (int i = 0; i < 3; i++) {
        int price = prices[i], qty = qtys[i], senior = seniors[i];
        int total = price * qty - price * qty * 15 / 100 * senior + price * qty * 5 / 100;
        int net   = total > 1000 ? total - 50 : total;
        printf("Bill %d -> Buggy total: %d, net: %d\n", i + 1, total, net);
    }
    return 0;
}
```

Observed Output: `Bill 1 -> Buggy total: 1080, net: 1030` | `Bill 2 -> Buggy total: 13, net: 13` | `Bill 3 -> Buggy total: 945, net: 945`. Every observed output matches our pre-execution prediction.

3. Identification of Bugs & Inquiry on Undefined Behaviour

- **Tax Calculation Logic Error:** GST (5%) was computed on the gross amount (`price * qty * 5 / 100`) instead of the post-discount amount. In Bill 1, GST is Rs 60 instead of Rs 51 (5% of 1020), overcharging senior citizens.
- **Integer Division Truncation:** In Bill 2, `15 * 5 / 100` evaluates to `75 / 100 = 0`, completely dropping the 5% GST on small items, while `225 / 100 = 2` truncates Rs 0.25 of discount.
- **Undefined Behaviour in `count = count++ + 1;`:** Under the C standard, modifying a scalar object (`count`) more than once without an intervening sequence point—or reading its value for a purpose other than determining the new value to be stored—triggers **Undefined Behaviour (UB)**. Both the assignment `=` and post-increment `++` write to `count` unsequenced. With `-Wall`, GCC emits: `warning: operation on 'count' may be undefined [-Wsequence-point]`.

4. Design Decision & Alternative Complexity Analysis

Design Decision: Use **both intermediate variables and explicit brackets** with `double` arithmetic. Intermediate variables eliminate repeated sub-expressions (`price * qty`), make the tax base unambiguous, and allow easy debugging, while brackets clarify operator grouping.

```
double gross    = (double)price * qty;
double discount = senior ? (gross * 0.15) : 0.0;
double taxable  = gross - discount;
double gst      = taxable * 0.05;
double total    = taxable + gst;
double net      = (total > 1000.0) ? (total - 50.0) : total;
```

Complexity Analysis: Replacing the single expression with intermediate variables reduces arithmetic operations from 9 (5 multiplications, 2 divisions, 2 additions/subtractions) down to 5 operations (`O(1)` time in both, but fewer CPU instructions). Extra space is 4 `double` variables (32 bytes on the stack, `O(1)` auxiliary space), which an optimizing compiler maps directly to CPU registers with zero memory overhead while guaranteeing financial accuracy and maintainability.

Q2 — When Does Short-Circuit Evaluation Matter?

1. Experimental Design & Call-Count Evidence

A concession is granted when `(is_senior && eligible_class) || verify_disability_server(id)`. In C, `&&` skips its right operand if the left operand is `0` (false); `||` skips its right operand if the left operand is non-zero (true). In contrast, bitwise `&` and `|` always evaluate both operands.

```
#include <stdio.h>

int server_calls = 0;

int verify_disability(int has_card) {
    server_calls++;
    return has_card;
}

int main(void) {
    /* Dataset of 1,000 passengers: 400 Senior+Eligible (40%),
       200 Senior+Ineligible (20%), 100 Non-Senior+Disabled (10%), 300 Regular (30%) */
    int senior[1000], eligible[1000], disabled[1000];
    for (int i = 0; i < 1000; i++) {
        senior[i]   = (i < 600);
        eligible[i] = (i < 400);
        disabled[i] = (i >= 600 && i < 700);
    }

    /* Order A: Local checks first, slow server check last */
    server_calls = 0;
    for (int i = 0; i < 1000; i++) {
        int grant = (senior[i] && eligible[i]) || verify_disability(disabled[i]);
        (void)grant;
    }
    printf("Order A (Local first with &&, ||): %d server calls\n", server_calls);

    /* Order B: Slow server check first */
    server_calls = 0;
    for (int i = 0; i < 1000; i++) {
        int grant = verify_disability(disabled[i]) || (senior[i] && eligible[i]);
        (void)grant;
    }
    printf("Order B (Server first with ||):      %d server calls\n", server_calls);

    /* Order C: Bitwise non-short-circuit (&, |) */
    server_calls = 0;
    for (int i = 0; i < 1000; i++) {
        int grant = (senior[i] & eligible[i]) | verify_disability(disabled[i]);
        (void)grant;
    }
    printf("Order C (Bitwise &, |):              %d server calls\n", server_calls);
    return 0;
}
```

Condition Ordering / Operator	Observed Server Calls (N = 1,000)	Explanation
Order A: `(senior && eligible) || verify_disability()`	**600 calls**	The 400 eligible seniors satisfy the left side of `||`; the slow server RPC is skipped for all 400.
Order B: `verify_disability() || (senior && eligible)`	**1,000 calls**	Placed as the left operand of `||`, the slow function executes unconditionally for every passenger.
Order C: `(senior & eligible) | verify_disability()`	**1,000 calls**	Bitwise `|` has no short-circuit semantics; both operands are strictly evaluated before bitwise OR.

2. Ordering Rules, Audit Log Impact & Alternative Complexity

- **When Ordering Matters vs. Irrelevant:** Place the *cheapest* or *most-likely-false* condition first in an `&&` chain (and *cheapest / most-likely-true* first in an `||` chain) to maximize short-circuit skipping. Ordering is irrelevant only when all operands are pure, constant-time local expressions (e.g., `a > 0 && b > 0`) with no side effects or latency difference.
- **Changed Condition (Mandatory Audit Log):** If `verify_disability()` writes an audit log required for *every* passenger, short-circuiting becomes a **correctness bug** because eligible seniors would bypass logging. In that case, `verify_disability()` must be called unconditionally before the `if` condition (or placed first in the expression) so all 1,000 passengers are logged.
- **Alternative & Complexity Analysis:** For n passengers with fraction p qualifying via senior+eligible status, nested `if-else` statements match short-circuiting with $n(1-p)$ server calls (`O(n)` time, lower constant factor). Using bitwise `&` and `|` always executes n server calls (worst-case network latency for every passenger) and breaks safety if the right operand guards against null pointers or division by zero.

Q3 — Engineer a Device-Status Register with Bitwise Operators

1. Completed Code & Execution Trace

```
#include <stdio.h>

enum { LIGHT = 0, FAN = 1, AC = 2, HEATER = 3, PUMP = 4 };

unsigned char switch_on(unsigned char reg, int dev)  { return reg | (1u << dev); }
unsigned char switch_off(unsigned char reg, int dev) { return reg & ~(1u << dev); }
unsigned char toggle(unsigned char reg, int dev)     { return reg ^ (1u << dev); }
int is_on(unsigned char reg, int dev)                { return (reg >> dev) & 1u; }

int count_on(unsigned char reg) {
    int count = 0;
    for (int b = 0; b < 8; b++) count += (reg >> b) & 1u;
    return count;
}

void show(unsigned char reg) {
    for (int bit = 7; bit >= 0; bit--) putchar(((reg >> bit) & 1u) ? '1' : '0');
    putchar('\n');
}

int main(void) {
    unsigned char reg = 0;
    reg = switch_on(reg, LIGHT);
    reg = switch_on(reg, AC);
    show(reg);
    reg = toggle(reg, FAN);
    reg = switch_off(reg, LIGHT);
    show(reg);
    printf("AC is %s, PUMP is %s\n", is_on(reg, AC) ? "ON" : "OFF",
                                     is_on(reg, PUMP) ? "ON" : "OFF");
    return 0;
}
```

Statement in `main()`	Predicted Binary	Predicted Decimal	Observed Result
`unsigned char reg = 0;`	`00000000`	0	0 (initial state)
`reg = switch_on(reg, LIGHT);` (bit 0)	`00000001`	1	1
`reg = switch_on(reg, AC);` (bit 2) → `show(reg)`	`00000101`	5	Prints `00000101`
`reg = toggle(reg, FAN);` (bit 1)	`00000111`	7	7
`reg = switch_off(reg, LIGHT);` (bit 0) → `show(reg)`	`00000110`	6	Prints `00000110`
`is_on(reg, AC)` & `is_on(reg, PUMP)`	Bit 2 = 1, Bit 4 = 0	1 and 0	Prints `AC is ON, PUMP is OFF`

2. Inquiry: Precedence & Signedness Pitfalls

- **`reg & 1 << dev` vs. `(reg & 1) << dev`:** In C, bit-shift `<<` has *higher precedence* than bitwise AND `&`. Thus, `reg & 1 << dev` parses as `reg & (1 << dev)`, isolating bit `dev` of `reg`. Conversely, `(reg & 1) << dev` masks bit 0 of `reg` (the LIGHT status) and shifts that single bit left by `dev` positions—completely ignoring device `dev`!
- **Using `signed char` when Bit 7 is Set:** In two's complement representation, bit 7 is the sign bit. Setting bit 7 makes the `signed char` negative (e.g., `-128`). Shifting a negative signed value left (`reg << 1`) is **Undefined Behaviour** in C, and right-shifting (`reg >> bit`) performs an *arithmetic shift* that sign-extends with `1`s rather than `0`s, corrupting bit masks and comparisons.

3. Alternative Complexity Analysis (`char state[8]` vs. Bitfield)

Storing states in `char state[8]` preserves `O(1)` time for `switch_on`, `switch_off`, `toggle`, and `is_on` via direct array indexing (`state[dev] = 1`). Counting active devices requires scanning all 8 elements (`O(w)` where $w = 8$ bits, i.e., `O(1)` fixed bound, though Brian Kernighan's algorithm `reg &= (reg - 1)` on a bit-register takes iterations equal only to the number of ON devices). For 1,000,000 homes, the bit-register uses **1,000,000 bytes (~0.95 MB)** whereas `char state[8]` consumes **8,000,000 bytes (~7.63 MB)**—an 8× memory reduction critical for constrained microcontrollers.

Q4 — else-if Ladder or switch?

1. Dual Implementation & Boundary Testing

```
#include <stdio.h>

/* Version 1: switch on marks / 10 */
char grade_switch(int marks) {
    if (marks < 0 || marks > 100) return '?';
    switch (marks / 10) {
        case 10:
        case 9:  return 'S';
        case 8:  return 'A';
        case 7:  return 'B';
        case 6:  return 'C';
        case 5:  return 'D';
        default: return 'F';
    }
}

/* Version 2: else-if ladder */
char grade_elseif(int marks) {
    if (marks < 0 || marks > 100) return '?';
    else if (marks >= 90) return 'S';
    else if (marks >= 80) return 'A';
    else if (marks >= 70) return 'B';
    else if (marks >= 60) return 'C';
    else if (marks >= 50) return 'D';
    else return 'F';
}

int main(void) {
    int test_cases[] = {-1, 0, 49, 50, 89, 90, 100, 101};
    for (int i = 0; i < 8; i++) {
        int m = test_cases[i];
        printf("Marks %3d -> switch: %c | else-if: %c\n", m, grade_switch(m),
grade_elseif(m));
    }
    return 0;
}
```

Test Input	`marks / 10`	`grade_switch`	`grade_elseif`	Boundary Purpose
`-1`	0 (guarded)	`?`	`?`	Lower out-of-range check (without guard, `-1/10 == 0` wrongly yields `'F'`!)
`0`, `49`	0, 4	`F`, `F`	`F`, `F`	Minimum valid score and upper boundary of `F`
`50`, `89`	5, 8	`D`, `A`	`D`, `A`	Lower boundary of `D` and upper boundary of `A`
`90`, `100`	9, 10	`S`, `S`	`S`, `S`	Lower and upper boundary of `S` (`case 10` falls through to `case 9`)

Test Input	`marks / 10`	`grade_switch`	`grade_elseif`	Boundary Purpose
`101`	10 (guarded)	`?`	`?`	Upper out-of-range check (without guard, `101/10 == 10` wrongly yields `'S'`!)

2. Fall-Through Behaviour, Range Limitations & Design Decisions

- **Effect of Removing `return` / `break`:** If `return 'A';` in `case 8:` is replaced by `g = 'A';` without a `break;`, execution falls through into `case 7:`, overwriting `g = 'B'`. All scores in 80–89 would erroneously receive grade `B`.
- **Why `switch` Cannot Test Arbitrary Ranges (e.g., 75–84):** Standard C `case` labels require compile-time integer constant expressions. Integer division `marks / 10` only partitions numbers into fixed multiples of 10 (70–79, 80–89). Because 75–84 spans two tens-digits, `switch (marks / 10)` cannot separate 74 from 75.
- **Ease of Modification:** The `else-if` ladder is far easier to modify when grade cutoffs shift to non-multiples of 10 (such as 85 for S or 75 for A), requiring only changing literal constants.
- **Scholarship Decision Structure:** A **2D Lookup Table** (`scholarship[grade_idx][income_band]`) is superior to nested `if` statements or a `switch` inside an `if`. It eliminates deep indentation, executes in deterministic `O(1)` time, and externalizes policy rules into declarative data.
- **Alternative Lookup Table Complexity:** Using `char grades[11] = {'F','F','F','F','F','D','C','B','A','S','S'};` achieves strict `O(1)` time (1 array index, vs. up to $k = 6$ branch comparisons in an `else-if` ladder) at a negligible space cost of 11 bytes.

Q5 — Count the Work Done by a Loop

1. Loop Derivation & Three Loop Implementations

Mathematical Principle: If $n = a \times b$ and $a > \sqrt{n}$, then $b = n / a < \sqrt{n}$. Thus, if no divisor exists in $[2, \sqrt{n}]$, n is prime. To avoid floating-point inaccuracy and `sqrt()` overhead, we square both sides of $i \le \sqrt{n}$ to get the integer loop condition `i * i <= n` (or overflow-safe `i <= n / i`).

```c
#include <stdio.h>

unsigned long checks = 0;

/* 1. Improved FOR loop */
int is_prime_for(unsigned long n) {
    checks = 0;
    if (n < 2) return 0;
    for (unsigned long i = 2; i <= n / i; i++) {
        checks++;
        if (n % i == 0) return 0;
    }
    return 1;
}

/* 2. Improved WHILE loop */
int is_prime_while(unsigned long n) {
    checks = 0;
    if (n < 2) return 0;
    unsigned long i = 2;
    while (i <= n / i) {
        checks++;
        if (n % i == 0) return 0;
        i++;
    }
    return 1;
}

/* 3. Improved DO-WHILE loop (requires explicit guard for n = 2 and n = 3) */
int is_prime_dowhile(unsigned long n) {
    checks = 0;
    if (n < 2) return 0;
    if (n == 2 || n == 3) return 1;
    unsigned long i = 2;
    do {
        checks++;
        if (n % i == 0) return 0;
        i++;
    } while (i <= n / i);
    return 1;
}
```

Ticket Number (n)	Status	Divisions (`i <= n / i`)	Divisions (`i < n`)	Speedup Factor
`97`	Prime	**8** (i = 2..9)	**95** (i = 2..96)	11.9× fewer
`100`	Not Prime	**1** (stops at i = 2)	**1** (stops at i = 2)	1.0× (early exit)
`7,919`	Prime	**87** (i = 2..88)	**7,917** (i = 2..7918)	91.0× fewer
`1,000,003`	Prime	**999** (i = 2..1000)	**1,000,001** (i = 2..1000002)	1,001× fewer

2. Inquiry & Sieve of Eratosthenes Complexity Analysis

- **Why `do-while` Needs Special Care for `n = 2, 3`:** A `do-while` loop executes its body *at least once* before checking the condition. For `n = 2`, `2 % 2 == 0` would execute and falsely return `0` (not prime). For `n = 3`, `3 % 2` performs an unnecessary division. Guarding `if (n == 2 || n == 3) return 1;` prevents both issues.
- **Integer Overflow in `i * i <= n`:** On systems with 32-bit `unsigned long` (max 4,294,967,295), if $n = 4{,}294{,}967{,}291$, when $i = 65{,}536$, `i * i` wraps around modulo 2^{32} to `0`, creating an infinite loop! Writing `i <= n / i` avoids multiplication overflow completely.
- **Sieve of Eratosthenes Alternative:** Precomputing primality up to N takes `O(N log log N)` preprocessing time and `O(N)` space (~1 MB for $N = 10^6$), after which every ticket lookup is `O(1)`. For a college fest checking thousands of tickets up to 1,000,003, the Sieve is vastly faster than `O(√n)` trial division per query.

Q6 — Loops That Must Terminate: Loan Repayment Schedule

1. Loop Design, Hand Verification & Safety Guard

A **`while` loop** (`while (balance > 1e-6)`) is the exact match because the number of months is unknown beforehand and zero iterations should run if the principal is already zero. Testing `balance != 0` is dangerous in IEEE-754 floating-point arithmetic because fractional interest percentages (e.g., `0.02`) cannot be represented exactly in binary, causing `balance` to miss exact zero (e.g., `-0.000000000001`) and loop infinitely.

```c
#include <stdio.h>

void simulate_loan(double principal, double rate, double emi) {
    double balance = principal, total_interest = 0.0;
    int month = 0;

    if (emi <= principal * rate) {
        printf("ERROR: EMI (%.2f) does not exceed first month interest (%.2f). Loan will
never terminate.\n",
               emi, principal * rate);
        return;
    }

    while (balance > 1e-6) {
        month++;
        if (month > 360) {
            printf("ALERT: Repayment exceeds 360-month safety limit. Increase monthly
EMI.\n");
            break;
        }
        double interest = balance * rate;
        double payment  = (balance + interest < emi) ? (balance + interest) : emi;
        balance = balance + interest - payment;
        total_interest += interest;
        printf("Month %2d | Interest: %6.2f | Payment: %7.2f | Closing Balance: %8.2f\n",
               month, interest, payment, balance);
    }
    if (month <= 360)
        printf("Summary: Repaid in %d months | Total Interest: Rs %.2f\n\n", month,
total_interest);
}

int main(void) {
    simulate_loan(10000.0, 0.02, 2000.0);
    return 0;
}
```

Month	Opening Balance	Interest (2%)	Actual Payment	Closing Balance	Verification
1	10,000.00	200.00	2,000.00	8,200.00	Matches hand calc
2	8,200.00	164.00	2,000.00	6,364.00	Matches hand calc
3	6,364.00	127.28	2,000.00	4,491.28	Matches hand calc
4	4,491.28	89.83	2,000.00	2,581.11	Matches hand calc
5	2,581.11	51.62	2,000.00	632.73	Matches hand calc
6	632.73	12.65	**645.38** (capped)	**0.00**	Total Interest: **Rs 645.38**

2. Growth Investigation, Deployment Decision & Closed-Form Analysis

- **Growth as Repayment Approaches Monthly Interest:** For Rs 10,000 at 2% (initial interest = Rs 200/month), EMI = Rs 2,000 takes **6 months**; EMI = Rs 500 takes **26 months**; EMI = Rs 250 takes **81 months**; EMI = Rs 205 takes **187 months**; EMI = Rs 201 takes **268 months**. As $EMI \to r \times P$, iterations grow logarithmically toward infinity.
- **Deployment Decision:** We combine an upfront solvency check (`emi <= principal * rate`) with a 360-month (30-year) loop guard. When triggered, the program outputs an actionable message explaining that the tenure exceeds 30 years and advises increasing the monthly repayment, protecting the server from CPU exhaustion.
- **Closed-Form Formula vs. Loop Simulation:** The analytical formula $n = -log(1 - r \cdot P / E) / log(1 + r)$ computes exact tenure in `O(1)` time and `O(1)` space without generating a schedule. However, for financial statements requiring per-month rounding to paise (cents) and an itemized passbook table, the `O(n)` simulation loop is mandatory because rounding each month slightly shifts the final payoff amount.

Q7 — Diagnose, Redesign and Validate

A. Diagnose the Bottleneck (Selected Program: Q5 Prime Ticket Verification)

In Q5, our evidence demonstrates two compounding performance bottlenecks when scaling ticket verification to high volumes (e.g., 100,000 tickets up to $N = 1{,}000{,}003$):

1. **Redundant Even-Divisor Testing:** Inside `is_prime()` , after testing `i = 2` , the loop still tests every even candidate (4, 6, 8, ..., 1000). For $n = 1{,}000{,}003$, 499 out of 999 divisions (50%) are wasted on even numbers that can never divide an odd integer.
2. **Repeated Primality Computation Across Queries:** Each call to `is_prime(n)` restarts trial division from scratch in `O(√n)` time. Across $Q = 100{,}000$ tickets, repeated queries perform up to $10^5 \times 999 \approx 10^8$ modulo operations, even when identical ticket numbers are checked repeatedly.

B. Define the Design Objective

Measurable Objectives:

- **Objective 1 (Single-Query Optimization):** Reduce worst-case trial divisions per ticket by **at least 66.7%** (cutting divisions for $n = 1{,}000{,}003$ from 999 down to ≤ 333) by exploiting the $6k \pm 1$ prime distribution property.
- **Objective 2 (Batch-Query Amortization):** Reduce amortized lookup cost across Q tickets up to $N = 10^6$ from `O(Q√N)` down to **`O(1)` per query** (zero divisions at lookup time) using a bit-packed Sieve of Eratosthenes inspired by Q3's register design.

C. Design the Revised Solution

Exploited Mathematical & Structural Properties:

- **6k ± 1 Wheel Factorization (for uncached/large numbers):** Every integer can be written as $6k + r$ where $r \in \{0, 1, 2, 3, 4, 5\}$. Multiples of 2 ($r = 0, 2, 4$) and multiples of 3 ($r = 0, 3$) are composite for $n > 3$. Thus, all primes > 3 take the form $6k - 1$ or $6k + 1$, allowing the loop to increment by 6 and test only two divisors per step.
- **Bit-Packed Sieve of Eratosthenes (for festival range $[0, N]$):** Combining Q5's Sieve with Q3's bit-level packing stores 8 primality flags per byte (requiring only `1,000,004 / 8 ≈ 125 KB` of RAM instead of 1 MB for a `char` array).
- **Trade-off:** Requires a one-time initialization pass of `O(N log log N)` time (~1.4 ms) and 125 KB of static memory, trading minimal RAM for instantaneous `O(1)` ticket verification.

```
ALGORITHM BitPackedSieveAndWheelPrime(N, query_list)
  // Phase 1: One-time Bit-Packed Sieve for [0..N] (125 KB memory)
  ALLOCATE bit_array[(N / 8) + 1] INITIALIZED TO 0xFF
  CLEAR_BIT(bit_array, 0); CLEAR_BIT(bit_array, 1)
  FOR i = 2 TO floor(sqrt(N)) DO
      IF GET_BIT(bit_array, i) == 1 THEN
          FOR j = i * i STEP i UP TO N DO
              CLEAR_BIT(bit_array, j)
          END FOR
      END IF
  END FOR

  // Phase 2: O(1) Lookup for in-range tickets; 6k +/- 1 fallback for n > N
  FOR EACH ticket IN query_list DO
      IF ticket <= N THEN
          RETURN GET_BIT(bit_array, ticket)          // O(1) time, 0 divisions
      ELSE
          IF ticket % 2 == 0 OR ticket % 3 == 0 THEN RETURN 0
          FOR k = 5 STEP 6 WHILE k <= ticket / k DO  // Tests only 6k-1 and 6k+1
              IF ticket % k == 0 OR ticket % (k + 2) == 0 THEN RETURN 0
          END FOR
          RETURN 1
      END IF
  END FOR
END ALGORITHM
```